- You have one place where you can easily add related features such as caching, metrics tracking, or logging.

Even though wrapping a dependency minimizes the risk of its interface changing from underneath you, this risk is still present. To address it, you can write a small number of integration specs that test your wrapper against the real library or service.

In the following example, we're testing an Invoice class that uses the TaxJar Ruby API client to calculate sales tax for a shopping site.[14] The initial version calls TaxJar directly, and then we'll refactor our class to use a wrapper. Here's the public interface of the class:

15-using-test-doubles-effectively/21/sales_tax/lib/invoice.rb

```
class Invoice
  def initialize(address, items, tax_client: MyApp.tax_client)
    @address = address
    @items = items
    @tax_client = tax_client
  end

  def calculate_total
    subtotal = @items.map(&:cost).inject(0, :+)
    taxes = subtotal * tax_rate
    subtotal + taxes
  end

  # ...
end
```

A new Invoice needs a shipping address and a list of items. To support testing, we're using dependency injection to pass a real or fake TaxJar implementation into the initializer. The calculate_total method tabulates the total cost of the items in the cart, and then applies the tax rate.

The tax rate lookup seems simple on the surface; once we know the ZIP code we're shipping to, we should be able to get the correct rate from TaxJar. The logic for doing so is a little convoluted, though:

15-using-test-doubles-effectively/22/sales_tax/lib/invoice.rb

```
def tax_rate
  @tax_client.rates_for_location(@address.zip).combined_rate
end
```

The TaxJar client requires more of us than a single method invocation. First, we call rates_for_location to get an object containing *all* the applicable tax rates (city, state, and so on). Then, we fetch its combined_rate to get the total sales tax.

14. https://github.com/taxjar/taxjar-ruby